Schema Supabase DBT AFKLM

Schéma Supabase et DBT pour AFKLM Flight Status aligné sur l'architecture afklm-delay-pipeline (repo prod)

Table des matières

1. Schéma Supabase et ingestion
   - **Normalisation** : Option A (JSONB) • Option B (tables normalisées)
   - **Ingestion** : dlt • Python
2. Structure DBT
3. Fichiers DBT à prévoir
4. Schémas Supabase
5. Schéma en étoile (Star Schema)
6. A confirmer avec Vincent

---

1. Schéma Supabase et ingestion

Les données brutes arrivent dans le schéma **`public`**. Deux structures possibles (Options A et B). L'ingestion peut être faite par dlt ou par un script Python (voir 1.3).

Dimension	Options
Normalisation (structure des données en base)	Option A (JSONB) ou Option B (tables normalisées)
Ingestion (qui charge les données)	dlt ou Python

Les options A et B définissent le **niveau de normalisation** des données : - **A** : données brutes en JSONB (pas de normalisation côté stockage) - **B** : données déjà normalisées en tables relationnelles (flights, legs, delays)

Option A : Table unique JSONB — données non normalisées (recommandée pour dbt)

```
-- public.operational_flights_raw
-- Charge brute par dlt (1 ligne = 1 réponse API ou 1 vol)
```

```sql
CREATE TABLE public.operational_flights_raw (
  id VARCHAR(50) PRIMARY KEY,           -- ex: "20260111+DL+5222"
  payload JSONB NOT NULL,               -- tout le vol (flight + legs)
  fetched_at TIMESTAMPTZ DEFAULT NOW(),
  source VARCHAR(50) DEFAULT 'afklm_flightstatus'
);

CREATE INDEX idx_operational_flights_raw_fetched ON
        public.operational_flights_raw(fetched_at);
CREATE INDEX idx_operational_flights_raw_payload ON
        public.operational_flights_raw USING GIN (payload);
```

Option B : Tables normalisées

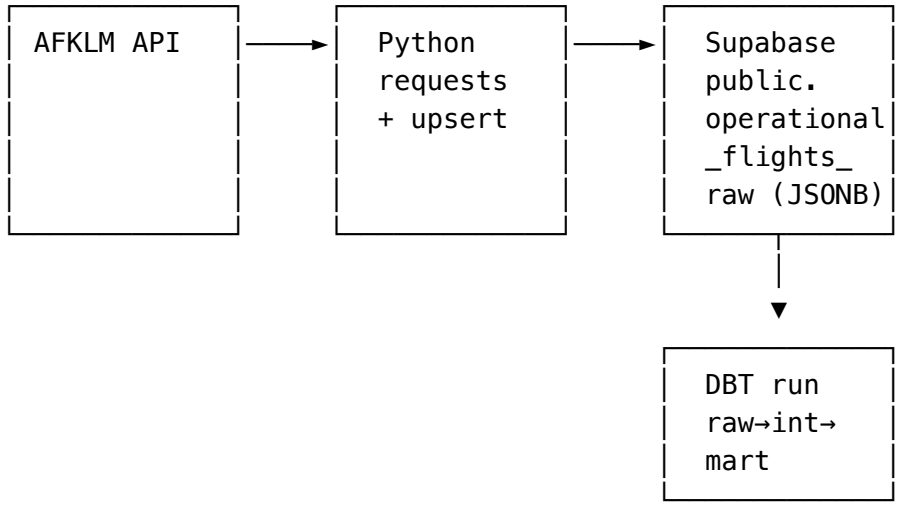
```sql
-- public.operational_flights
CREATE TABLE public.operational_flights (
  id VARCHAR(50) PRIMARY KEY,
  flight_number INTEGER,
  flight_schedule_date DATE,
  airline_code VARCHAR(3),
  airline_name TEXT,
  haul VARCHAR(20),
  route TEXT[],
  flight_status_public VARCHAR(50),
  fetched_at TIMESTAMPTZ DEFAULT NOW()
);

-- public.operational_flight_legs
CREATE TABLE public.operational_flight_legs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  flight_id VARCHAR(50) REFERENCES public.operational_flights(id),
  leg_order INTEGER,
  departure_airport_code VARCHAR(3),
  arrival_airport_code VARCHAR(3),
  published_status VARCHAR(50),
  scheduled_departure TIMESTAMPTZ,
  actual_departure TIMESTAMPTZ,
  scheduled_arrival TIMESTAMPTZ,
  actual_arrival TIMESTAMPTZ,
  scheduled_flight_duration VARCHAR(20),
  cancelled BOOLEAN DEFAULT FALSE
);

-- public.operational_flight_delays
CREATE TABLE public.operational_flight_delays (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  flight_leg_id UUID REFERENCES public.operational_flight_legs(id),
  delay_code VARCHAR(10),
  delay_duration VARCHAR(10)
);
```

1.3 Sans dlt : Python requests

Avec Python : l'approche est d'utiliser **Option A (JSONB)**. Le script reste minimal (fetch + upsert), et DBT conserve toute la logique de transformation.



Exemple de script Python (Option A) :

```python
import requests
from datetime import datetime, timezone
from supabase import create_client

response = requests.get(api_url, headers={"API-Key": key})
data = response.json()

for flight in data["operationalFlights"]:
    supabase.table("operational_flights_raw").upsert({
        "id": flight["id"],
        "payload": flight,  # JSONB direct
        "fetched_at": datetime.now(timezone.utc).isoformat(),
    }).execute()
```

Responsabilités : Python fait uniquement E + L (extract + load). DBT fait T (transform). À éviter : parser les legs, normaliser en plusieurs tables, calculer les retards — tout cela reste dans DBT.

Planification : `python fetch_afklm.py && dbt run` (cron ou script bash).

---

2. Structure DBT (comme afklm-delay-pipeline - la prod)

Structure de base. La section 5 détaille le schéma en étoile complet.

```
models/
├── 1_raw/flight_data/
│   ├── sources.yml
```

```
│   └── flight_data__source_operational_flights.sql
├── 2_int/flight_data/
│   ├── flight_data__int_delays_leg.sql
│   └── properties.yml
└── 3_mart/delays/
    ├── dim_airlines.sql
    ├── dim_airports.sql
    ├── dim_date.sql
    ├── fct_flight_legs.sql
    ├── properties.yml
    └── exposures.yml
```

---

3. Fichiers DBT à prévoir

1_raw/flight_data/sources.yml

```yaml
version: 2

sources:
  - name: flight_data
    database: "{{ env_var('AFKLM_DB_NAME') }}"
    schema: public
    loaded_at_field: fetched_at
    description: 'Données brutes AFKLM Flight Status API (dlt)'
    tables:
      - name: operational_flights_raw   # ou operational_flights si Option B
        description: 'Vols opérationnels avec legs et irrégularités'
        freshness:
          warn_after: {count: 1, period: day}
          error_after: {count: 3, period: day}
```

1_raw/flight_data/flight_data__source_operational_flights.sql

Pour l'Option A (JSONB), dépliage du JSON :

```sql
-- raw.flight_data__source_operational_flights
-- Vue source : dépliage du JSONB vers structure exploitable
{{ config(schema='raw', materialized='view') }}

with base as (
    select
        id,
        payload,
        fetched_at
    from {{ source('flight_data', 'operational_flights_raw') }}
),
```

```sql
flights as (
    select
        payload->>'id' as flight_id,
        (payload->>'flightNumber')::int as flight_number,
        (payload->>'flightScheduleDate')::date as
        flight_schedule_date,
        payload->>'haul' as haul,
        payload->'route' as route,
        payload->'airline'->>'code' as airline_code,
        payload->'airline'->>'name' as airline_name,
        payload->>'flightStatusPublic' as flight_status_public,
        jsonb_array_elements(payload->'flightLegs') as leg,
        fetched_at
    from base
)
select
    flight_id,
    flight_number,
    flight_schedule_date,
    haul,
    route,
    airline_code,
    airline_name,
    flight_status_public,
    leg->'departureInformation'->'airport'->>'code' as
        departure_airport_code,
    leg->'arrivalInformation'->'airport'->>'code' as
        arrival_airport_code,
    leg->>'publishedStatus' as leg_status,
    (leg->'departureInformation'->'times'->>'scheduled')::timestamptz
        as scheduled_departure,
    (leg->'departureInformation'->'times'->>'actual')::timestamptz as
        actual_departure,
    (leg->'arrivalInformation'->'times'->>'scheduled')::timestamptz as
        scheduled_arrival,
    (leg->'arrivalInformation'->'times'->>'actual')::timestamptz as
        actual_arrival,
    leg->>'scheduledFlightDuration' as scheduled_flight_duration,
    (leg->'irregularity'->>'cancelled') = 'Y' as cancelled,
    leg->'irregularity'->'delayInformation' as delay_info,
    fetched_at
from flights
```

2_int/flight_data/flight_data__int_delays_leg.sql

```sql
-- int.flight_data__int_delays_leg
-- Grain : 1 ligne = 1 leg de vol avec indicateurs de retard
{{ config(schema='int', materialized='view') }}

with source as (
    select * from {{ ref('flight_data__source_operational_flights') }}
),
```

```sql
-- Legs avec au moins un retard (explosion 1 ligne par delay)
legs_with_delays as (
    select
        s.flight_id, s.flight_number, s.flight_schedule_date,
        s.airline_code, s.airline_name,
        s.departure_airport_code, s.arrival_airport_code,
        s.scheduled_departure, s.actual_departure,
        s.scheduled_arrival, s.actual_arrival,
        s.cancelled,
        d->>'delayCode' as delay_code,
        d->>'delayDuration' as delay_duration_minutes
    from source s,
    lateral jsonb_array_elements(s.delay_info) as d
    where jsonb_array_length(coalesce(s.delay_info, '[]'::jsonb)) > 0
),
-- Legs sans retard (1 ligne par leg)
legs_without_delays as (
    select
        flight_id, flight_number, flight_schedule_date,
        airline_code, airline_name,
        departure_airport_code, arrival_airport_code,
        scheduled_departure, actual_departure,
        scheduled_arrival, actual_arrival,
        cancelled,
        null::text as delay_code,
        null::text as delay_duration_minutes
    from source
    where jsonb_array_length(coalesce(delay_info, '[]'::jsonb)) = 0
),
unioned as (
    select * from legs_with_delays
    union all
    select * from legs_without_delays
)
select
    flight_id,
    flight_number,
    flight_schedule_date,
    airline_code,
    airline_name,
    departure_airport_code,
    arrival_airport_code,
    scheduled_departure,
    actual_departure,
    scheduled_arrival,
    actual_arrival,
    cancelled,
    delay_code,
    delay_duration_minutes,
    extract(epoch from (actual_departure - scheduled_departure)) / 60
        as departure_delay_minutes,
```

```
    extract(epoch from (actual_arrival - scheduled_arrival)) / 60 as
        arrival_delay_minutes
from unioned
```

Modèles mart (schéma en étoile)

Les modèles `dim_airlines`, `dim_airports`, `dim_date` et `fct_flight_legs` sont détaillés dans la **section 5.4**.

---

4. Schémas Supabase (alignés avec le README afklm-delay-pipeline)

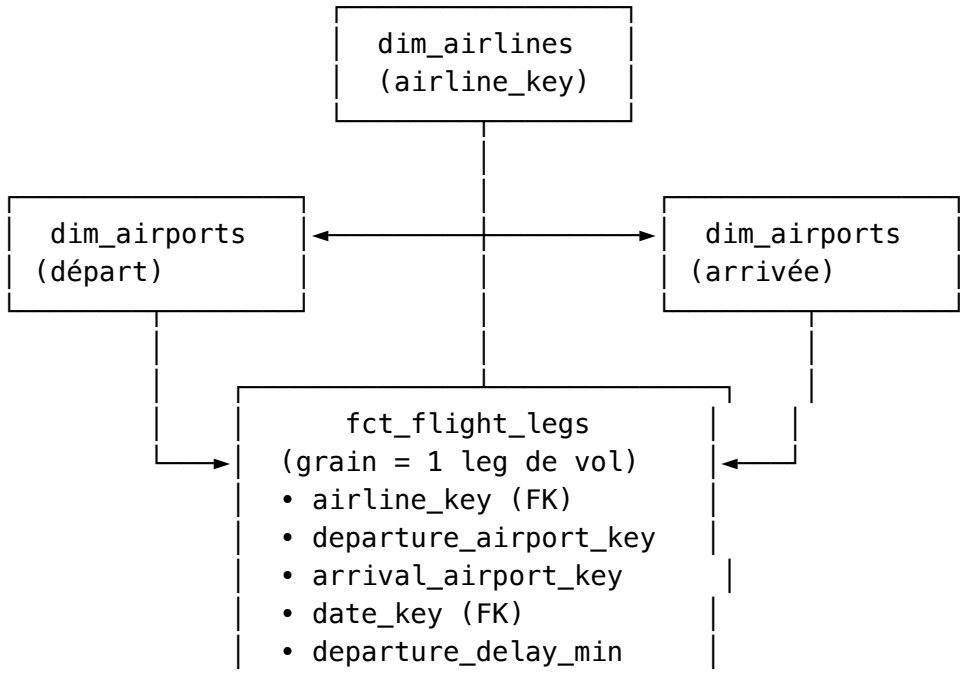
Schéma	Contenu
`public`	Tables brutes chargées par dlt
`raw`	Modèles source DBT (`flight_data__source_*`)
`int`	Modèles intermédiaires DBT
`mart`	`fct_flight_legs`, `dim_*` pour Metabase

---

5. Schéma en étoile (Star Schema)

Les **deux options** (A et B) peuvent alimenter le même schéma en étoile. Le schéma en étoile se définit dans la couche **mart** (3_mart) de DBT, indépendamment du format des données brutes. Seule la couche raw/public change selon l'option choisie.

5.1 Vue d'ensemble



```
│  • arrival_delay_min       │
└──────────────────────────┘
                │
                │
        ┌──────────────┐
        │   dim_date   │
        │  (date_key)  │
        └──────────────┘
```

5.2 Tables du schéma en étoile

Table	Rôle	Grain
fct_flight_legs	Fait central — mesures de retard et événements	1 ligne = 1 leg de vol
dim_airlines	Compagnie aérienne	1 ligne = 1 compagnie
dim_airports	Aéroport (départ et arrivée partagent la même dimension)	1 ligne = 1 aéroport
dim_date	Calendrier pour analyses temporelles	1 ligne = 1 jour

5.3 Flux selon l'option choisie

Couche	Option A (JSONB)	Option B (n
public	`operational_flights_raw` (JSONB)	`operational_flights` + `operational_flight_l` `operational_flight_d`
raw	`flight_data__source_operational_flights` (dépliage JSON)	`flight_data__source_` (SELECT sur les tables)
int	`flight_data__int_delays_leg`	`flight_data__int_del`
mart	**Même schéma en étoile**	**Même schéma en étoile**

La couche mart est identique dans les deux cas : les dimensions et le fait sont construits à partir du modèle int commun.

5.4 Modèles mart en schéma en étoile

Dimensions

dim_airlines — une ligne par compagnie :

```
-- mart.dim_airlines
{{ config(schema='mart', materialized='table') }}

select distinct
    airline_code as airline_key,   -- PK (code IATA = clé naturelle)
    airline_code,
    airline_name
```

```
from {{ ref('flight_data__int_delays_leg') }}
where airline_code is not null
```

dim_airports — une ligne par aéroport (départ et arrivée) :

```
-- mart.dim_airports
{{ config(schema='mart', materialized='table') }}

with airports_union as (
    select departure_airport_code as airport_code from {{
        ref('flight_data__int_delays_leg') }}
    union
    select arrival_airport_code from {{
        ref('flight_data__int_delays_leg') }}
)
select distinct
    airport_code as airport_key,   -- PK
    airport_code
from airports_union
where airport_code is not null
```

dim_date — calendrier (peut être une seed ou un modèle) :

```
-- mart.dim_date
{{ config(schema='mart', materialized='table') }}

with date_range as (
    select generate_series(
        (select min(flight_schedule_date) from {{
        ref('flight_data__int_delays_leg') }}),
        (select max(flight_schedule_date) from {{
        ref('flight_data__int_delays_leg') }}),
        '1 day'::interval
    )::date as date_key
)
select
    date_key,
    extract(dow from date_key) as day_of_week,
    extract(month from date_key) as month,
    extract(year from date_key) as year,
    to_char(date_key, 'TMDay') as day_name
from date_range
```

Fait avec clés étrangères

fct_flight_legs — fait central avec FK vers les dimensions :

```
-- mart.fct_flight_legs
-- Schéma en étoile : FK vers dim_airlines, dim_airports (x2),
        dim_date
{{ config(schema='mart', materialized='table') }}

select
```

```
    i.flight_id,
    i.flight_number,
    i.airline_code as airline_key,              -- FK → dim_airlines
    i.departure_airport_code as departure_airport_key,  -- FK →
        dim_airports
    i.arrival_airport_code as arrival_airport_key,    -- FK →
        dim_airports
    i.flight_schedule_date as date_key,         -- FK → dim_date
    i.scheduled_departure,
    i.actual_departure,
    i.scheduled_arrival,
    i.actual_arrival,
    i.cancelled,
    i.delay_code,
    i.delay_duration_minutes,
    i.departure_delay_minutes,
    i.arrival_delay_minutes
from {{ ref('flight_data__int_delays_leg') }} i
```

> **Note :** Ici on utilise les codes IATA (`airline_code`, `airport_code`) comme clés naturelles. Pour des clés de substitution (surrogate keys), on peut ajouter `row_number() over (order by ...)` dans les dimensions.

5.5 Structure DBT mise à jour (schéma en étoile)

```
models/
├── 1_raw/flight_data/
│   ├── sources.yml
│   └── flight_data__source_operational_flights.sql
├── 2_int/flight_data/
│   ├── flight_data__int_delays_leg.sql
│   └── properties.yml
└── 3_mart/delays/
    ├── dim_airlines.sql
    ├── dim_airports.sql
    ├── dim_date.sql
    ├── fct_flight_legs.sql
    ├── properties.yml
    └── exposures.yml
```

---

6. A confirmer avec Vincent :

- **Option A (JSONB)** : plus simple pour dlt (charge le JSON tel quel), DBT fait le dépliage dans `flight_data__source_operational_flights`.
- **Option B (tables normalisées)** : plus de travail côté dlt (flattening), mais DBT plus simple.

Pour rester cohérent avec afklm-delay-pipeline, l'**Option A** serait point de départ : une table `public.operational_flights_raw` avec JSONB, puis DBT pour raw → int → mart jusqu'au schéma en étoile `mart.fct_flight_legs` + `mart.dim_*` consommé par

Metabase et `ml_score.py` ??